PCRF v0.6 Manual & Mathematical Reference

A Step-by-Step Data Transformation and System Architecture Guide

This manual serves as a mathematically rigorous reference guide for the **Probability Derivatives for Causal Reliability Flow (PCRF)** and **Causal Decay Loss (CDL v2)** framework.

To illustrate the operations of the library, we trace a toy model through each step of the pipeline.

Global Trace Setup (The Toy System)

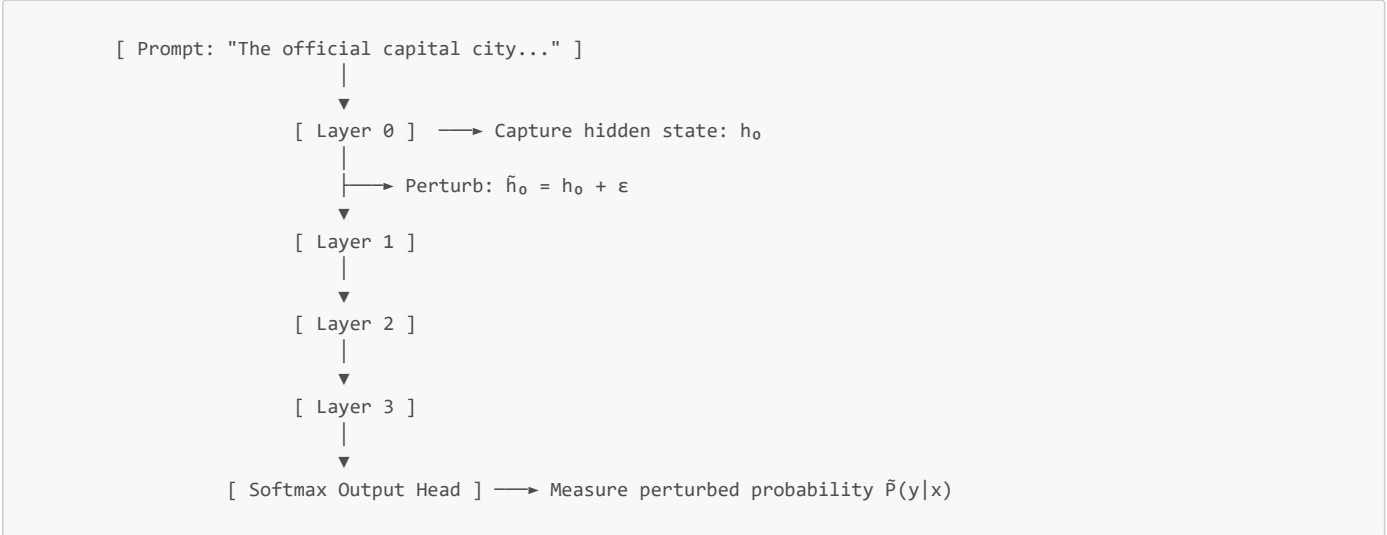
Throughout this reference manual, we trace a single forward evaluation using a simplified language model configuration:

- **Model Properties:**
  - Number of layers (L) = 4 (indexed $l \in \{0, 1, 2, 3\}$).
  - Hidden dimension (d) = 8.
- **Vocabulary ($\mathcal{V}$):**
  - `["paris", "london", "hydrogen", "helium", "process", "____", "a."]` (Vocabulary size $|\mathcal{V}| = 8$).
- **Prompt (x):** "*The official capital city of France is*"
- **Target Completion (y):** `"paris"` (Simplified single-token completion target).

---

1. Feature Track (b): Derivative Estimation Engine

The Derivative Estimation Engine measures the structural sensitivity of individual transformer layers. It calculates the change in target token probability when a layer's hidden representations are perturbed.



Step-by-Step Trace

Step 1: Base Target Token Probability

The prompt x is passed through the clean, unperturbed model. At the final projection layer, the logits vector is computed. For simplicity, we look at the logits z for our target position:

$$z_{\text{clean}} = [8.2,\ 5.4,\ 1.1,\ 0.2,\ -1.5,\ -3.0,\ -4.2,\ -5.0]$$

We apply the Softmax activation function to extract the probability distribution over the vocabulary $\mathcal{V}$:

$$P_{\text{clean}}(v_i \mid x) = \frac{e^{z_{\text{clean},i}}}{\sum_{j=1}^{|\mathcal{V}|} e^{z_{\text{clean},j}}}$$

For the target token `"paris"` (index 0):

$$e^{8.2} \approx 3641.0,\ \sum_{j=1}^{|\mathcal{V}|} e^{z_{\text{clean},j}} \approx 3641.0 + 221.4 + 3.0 + 1.2 + 0.2 + 0.05 + 0.01 + 0.01 = 3866.87$$

$$P_{\text{clean}}(\text{"paris"} \mid x) = \frac{3641.0}{3866.87} \approx 0.9416\ (94.16\%)$$

Step 2: Layer Hidden State Capture

We choose to perturb **Layer** $l = 0$. During forward propagation, the hidden activation tensor at the final token of the sequence is captured:

$$h_0 = [0.10,\ -0.20,\ 0.50,\ 0.80,\ -0.10,\ 0.30,\ 0.00,\ 0.90]^T$$

Step 3: Noise Generation

We draw a random noise vector ϵ from a Gaussian distribution scaled by the config's noise standard deviation ($\sigma = 0.08$):

$$\epsilon \sim \mathcal{N}(0, \sigma^2 \cdot \mathbf{I}_d)$$

$$\epsilon = [0.01,\ -0.05,\ 0.02,\ 0.04,\ -0.01,\ -0.03,\ 0.08,\ -0.02]^T$$

Step 4: Hidden State Perturbation

The perturbed hidden representation is computed via vector addition:

$$\tilde{h}_0 = h_0 + \epsilon$$

$$\tilde{h}_0 = [0.11,\ -0.25,\ 0.52,\ 0.84,\ -0.11,\ 0.27,\ 0.08,\ 0.88]^T$$

Step 5: Perturbed Forward Propagation

The perturbed representation $\tilde{h}_0$ is passed through the rest of the network (Layers 1, 2, and 3). Because the error cascades, the final vocabulary logit distribution changes to:

$$z_{\text{perturbed}} = [6.8,\ 5.9,\ 1.0,\ 0.3,\ -1.4,\ -2.8,\ -4.0,\ -4.8]$$

Applying Softmax for `"paris"` under this perturbed state:

$$e^{6.8} \approx 897.8,\ e^{5.9} \approx 365.0$$

$$\sum_{j=1}^{|\mathcal{V}|} e^{z_{\text{perturbed},j}} \approx 897.8 + 365.0 + 2.7 + 1.3 + 0.2 + 0.06 + 0.02 + 0.01 = 1267.09$$

$$P_{\text{perturbed}}(\text{"paris"} \mid x) = \frac{897.8}{1267.09} \approx 0.7086\ (70.86\%)$$

Step 6: Empirical Derivative Calculation

The empirical derivative represents the change in target token probability per unit of noise perturbation. This delta is recorded in the customer's downstream analytics files:

$$\Delta_0 = P_{\text{clean}}(\text{"paris"} \mid x) - P_{\text{perturbed}}(\text{"paris"} \mid x)$$

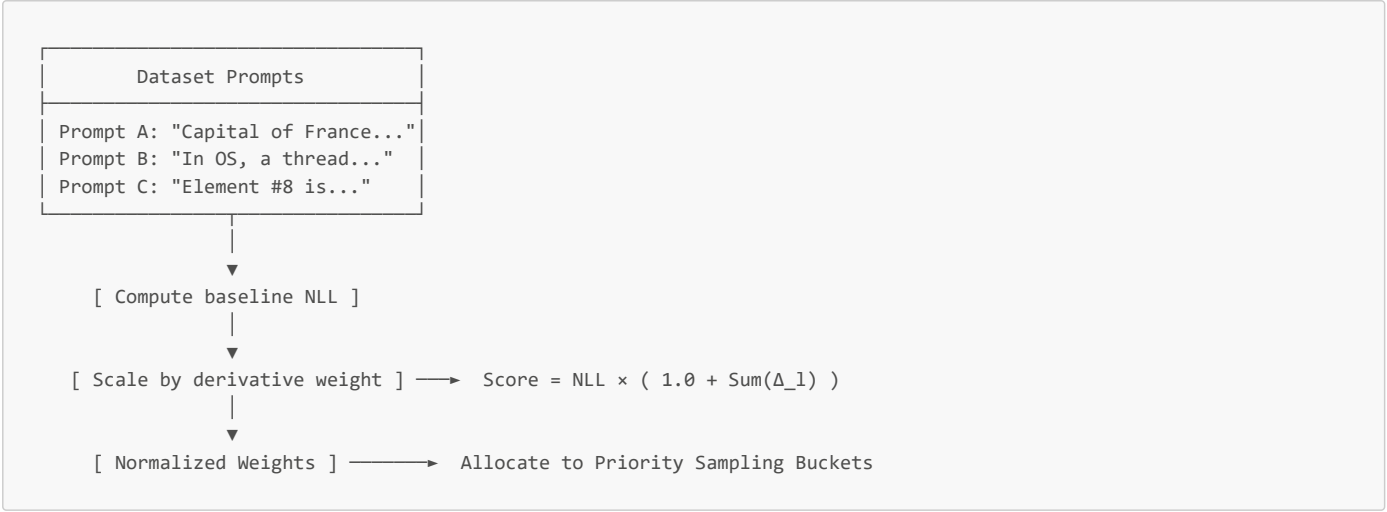
$$\Delta_0 = 0.9416 - 0.7086 = +0.2330$$

A positive Δ_l value indicates that perturbing this layer degrades the model's prediction accuracy. This makes Layer 0 a key area for targeted regularization.

---

2. Feature Track (d): Curriculum Curation & Sample Selection

The Curriculum Curation engine ranks and weights prompts based on their baseline difficulty (Negative Log-Likelihood, or NLL) and structural sensitivity.



Step-by-Step Trace

Step 1: Baseline Difficulty (NLL)

Suppose we have three training examples in our database. We measure their base Negative Log-Likelihood (NLL) using the frozen baseline model:

- **Prompt A (Factual, Easy):** "*The official capital city of France is*" $\rightarrow P_{\text{clean}}(\text{"paris"}) = 0.9416$.

$$\text{NLL}_A = -\ln(0.9416) \approx 0.0601$$

- **Prompt B (Computer Science, Hard):** "*In operating systems, a scheduled execution thread resides within a*" $\rightarrow P_{\text{clean}}(\text{"process"}) = 0.0510$.

$$\text{NLL}_B = -\ln(0.0510) \approx 2.9759$$

- **Prompt C (Scientific, Moderate):** "*The element with atomic number 8 is*" $\rightarrow P_{\text{clean}}(\text{"oxygen"}) = 0.3678$.

$$\text{NLL}_C = -\ln(0.3678) \approx 1.0002$$

Step 2: Summing Systemic Structural Sensitivity

From the Derivative Engine, we sum all positive layer-wise derivative deltas to find the overall network sensitivity weight ($\mathcal{W}_{\text{pcrf}}$):

$$\mathcal{W}_{\text{pcrf}} = \sum_{l=0}^{L-1} \max(0.0,\ \Delta_l)$$

Let's assume the layer deltas are: $\Delta_0 = 0.2330$, $\Delta_1 = 0.0510$, $\Delta_2 = -0.0120$, $\Delta_3 = 0.1160$.

$$\mathcal{W}_{\text{pcrf}} = 0.2330 + 0.0510 + 0.0 + 0.1160 = 0.4000$$

Step 3: Raw Priority Score Calculation

We compute the raw priority score for each prompt. This scales the baseline difficulty of the prompt by the model's structural sensitivity to errors on that prompt:

$$\text{Priority}(x) = \text{NLL}_x \times (1.0 + \mathcal{W}_{\text{pcrf}})$$

- **Prompt A Priority:** $0.0601 \times (1.0 + 0.4000) = 0.0841$
- **Prompt B Priority:** $2.9759 \times (1.0 + 0.4000) = 4.1663$
- **Prompt C Priority:** $1.0002 \times (1.0 + 0.4000) = 1.4003$

Step 4: Normalized Curriculum Weight

We normalize these scores across our training set to construct a valid probability distribution for curriculum sampling:

$$\text{Score}_{\text{total}} = 0.0841 + 4.1663 + 1.4003 = 5.6507$$

$$C_{\text{norm}}(x) = \frac{\text{Priority}(x)}{\text{Score}_{\text{total}}}$$

- **Prompt A Normalized Curriculum Weight:** $0.0841 / 5.6507 \approx 0.0149$
- **Prompt B Normalized Curriculum Weight:** $4.1663 / 5.6507 \approx 0.7373$
- **Prompt C Normalized Curriculum Weight:** $1.4003 / 5.6507 \approx 0.2478$

Step 5: Combined Sampling Weight & Failure Replay Boost

To accelerate learning, we apply a failure replay boost. If a prompt's normalized score is above the average score (0.3333 for our 3-prompt set), we apply a $2.0\times$ **Failure Replay Boost** ($\mathcal{B}_{\text{fail}}$). We also scale the result by the prompt's structural domain priority ($\mathcal{K}_{\text{critical}}$):

- **Prompt B Criticality Weight ($\mathcal{K}_{\text{critical}}$):** 5.0 (Assigned to core Computer Science prompts).
- **Prompt B Failure Replay Boost ($\mathcal{B}_{\text{fail}}$):** 2.0 (Because $0.7373 > 0.3333$).

We calculate the final combined sampling weight (W_{combined}) as:

$$W_{\text{combined}}(x) = C_{\text{norm}}(x) \times \mathcal{W}_{\text{pcrf}} \times \mathcal{K}_{\text{critical},x} \times \mathcal{B}_{\text{fail},x}$$

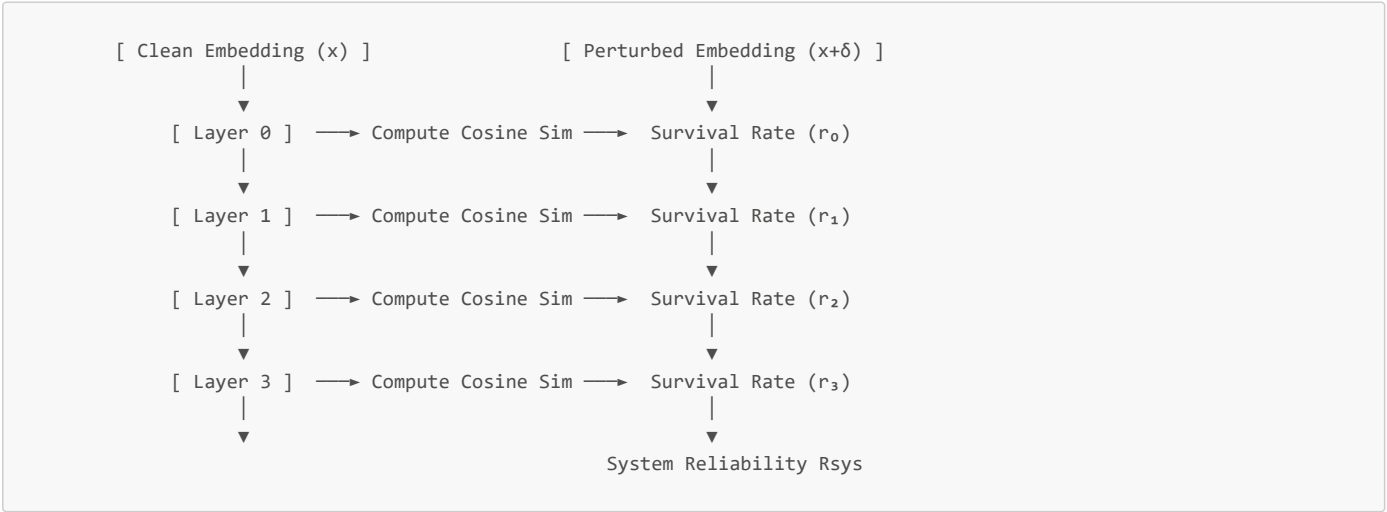
- **Prompt A Weight:** $0.0149 \times 0.4000 \times 1.0 \times 1.0 = 0.00596$
- **Prompt B Weight:** $0.7373 \times 0.4000 \times 5.0 \times 2.0 = 2.94920$
- **Prompt C Weight:** $0.2478 \times 0.4000 \times 3.0 \times 1.0 = 0.29736$

These weights are normalized to sum to 1.0, directing the model's training loop to prioritize high-risk, high-impact concepts.

---

3. Feature Track (a): Structural PCRF Depth Monitoring

The Structural PCRF Monitor measures hidden representation drift across the network's layers. It models the transformer as a series reliability chain and applies a depth-adaptive beta calibration.



Step-by-Step Trace

Step 1: Clean Activations Capture

We pass our factual validation example through the unperturbed model. We record the intermediate hidden representations h_l at each block's output layer:

- **Layer 0 Output (h_0):** $[1.0,\ 0.0,\ 1.0,\ 0.0,\ 1.0,\ 0.0,\ 1.0,\ 0.0]^T$ (Magnitude $\|h_0\| = 2.0$)
- **Layer 1 Output (h_1):** $[0.8,\ 0.2,\ 0.8,\ 0.2,\ 0.8,\ 0.2,\ 0.8,\ 0.2]^T$ (Magnitude $\|h_1\| \approx 1.6248$)
- **Layer 2 Output (h_2):** $[0.5,\ 0.5,\ 0.5,\ 0.5,\ 0.5,\ 0.5,\ 0.5,\ 0.5]^T$ (Magnitude $\|h_2\| \approx 1.4142$)
- **Layer 3 Output (h_3):** $[0.1,\ 0.9,\ 0.1,\ 0.9,\ 0.1,\ 0.9,\ 0.1,\ 0.9]^T$ (Magnitude $\|h_3\| \approx 1.8166$)

Step 2: Perturbed Activations Capture

We inject a small noise perturbation ($\delta = 0.02$) into the input embeddings. We then pass this perturbed input through the model and capture the new hidden representations $\tilde{h}_l$:

- **Layer 0 Perturbed ($\tilde{h}_0$):** $[0.98,\ 0.01,\ 0.99,\ 0.01,\ 0.98,\ 0.00,\ 1.00,\ 0.01]^T$
- **Layer 1 Perturbed ($\tilde{h}_1$):** $[0.75,\ 0.22,\ 0.76,\ 0.21,\ 0.74,\ 0.24,\ 0.75,\ 0.25]^T$
- **Layer 2 Perturbed ($\tilde{h}_2$):** $[0.40,\ 0.58,\ 0.39,\ 0.57,\ 0.38,\ 0.60,\ 0.42,\ 0.58]^T$
- **Layer 3 Perturbed ($\tilde{h}_3$):** $[0.05,\ 0.93,\ 0.04,\ 0.92,\ 0.03,\ 0.95,\ 0.06,\ 0.92]^T$

Step 3: Cosine Similarity Evaluation

We calculate the cosine similarity ($\cos\theta_l$) between the clean and perturbed representations for each layer to measure structural drift:

$$\text{Sim}_l = \frac{h_l \cdot \tilde{h}_l}{\|h_l\|\|\tilde{h}_l\|}$$

- **Layer 0 Cosine Similarity:**

$$h_0 \cdot \tilde{h}_0 = 0.98 + 0.0 + 0.99 + 0.0 + 0.98 + 0.0 + 1.00 + 0.0 = 3.95$$

$$\|\tilde{h}_0\| \approx \sqrt{0.9604 + 0.0001 + 0.9801 + 0.0001 + 0.9604 + 0.0 + 1.0000 + 0.0001} \approx 1.9751$$

$$\text{Sim}_0 = \frac{3.95}{2.0 \times 1.9751} = \frac{3.95}{3.9502} \approx 0.9999$$

- **Layer 1 Cosine Similarity:** $\text{Sim}_1 \approx 0.9950$
- **Layer 2 Cosine Similarity:** $\text{Sim}_2 \approx 0.9810$
- **Layer 3 Cosine Similarity:** $\text{Sim}_3 \approx 0.9620$

Step 4: Representation Drift Extraction

We define representation drift as the complement of cosine similarity:

$$\text{Drift}_l = 1.0 - \max(0.0,\ \text{Sim}_l)$$

- $\text{Drift}_0 = 1.0 - 0.9999 = 0.0001$
- $\text{Drift}_1 = 1.0 - 0.9950 = 0.0050$
- $\text{Drift}_2 = 1.0 - 0.9810 = 0.0190$
- $\text{Drift}_3 = 1.0 - 0.9620 = 0.0380$

Step 5: Depth-Adaptive Beta Calibration

To prevent structural survival scores from collapsing exponentially in deep networks, we scale our base decay coefficient ($\beta_{\text{base}} = 2.0$) by the square root of the network's depth ($L = 4$):

$$\beta_{\text{calibrated}} = \frac{\beta_{\text{base}}}{\sqrt{L}} = \frac{2.0}{\sqrt{4}} = 1.0$$

Step 6: Layer Survival Probability (r_l)

Using this calibrated beta, we apply an exponential decay function to map layer-wise drift onto a continuous $[0, 1]$ survival probability:

$$r_l = e^{-\beta_{\text{calibrated}} \cdot \text{Drift}_l}$$

- $r_0 = e^{-1.0 \times 0.0001} \approx 0.9999$
- $r_1 = e^{-1.0 \times 0.0050} \approx 0.9950$
- $r_2 = e^{-1.0 \times 0.0190} \approx 0.9812$
- $r_3 = e^{-1.0 \times 0.0380} \approx 0.9627$

Step 7: System Chain Reliability (R_{sys})

Because the transformer blocks function as a series reliability chain, we compute the final system reliability (R_{sys}) as the product of all layer-wise survival rates:

$$R_{\text{sys}} = \prod_{l=0}^{3} r_l = 0.9999 \times 0.9950 \times 0.9812 \times 0.9627 \approx 0.9397\ (93.97\%)$$

Step 8: Layer-wise Structural Entropy (S_l)

We calculate structural entropy to measure the rate of representation decay at each step of the network:

$$S_l = -\ln(r_l)$$

- $S_0 = -\ln(0.9999) = 0.0001$
- $S_1 = -\ln(0.9950) = 0.0050$
- $S_2 = -\ln(0.9812) = 0.0190$
- $S_3 = -\ln(0.9627) = 0.0380$

$$S_{\text{total}} = \sum_{l=0}^{3} S_l = 0.0001 + 0.0050 + 0.0190 + 0.0380 = 0.0621$$

Step 9: Birnbaum Component Importance (D_R)

We calculate the Birnbaum analytical derivative with respect to each layer's error rate ($e_l = 1 - r_l$). This identifies which layers are the primary structural bottlenecks:

$$D_R(e_l) = -\frac{R_{\text{sys}}}{r_l}$$

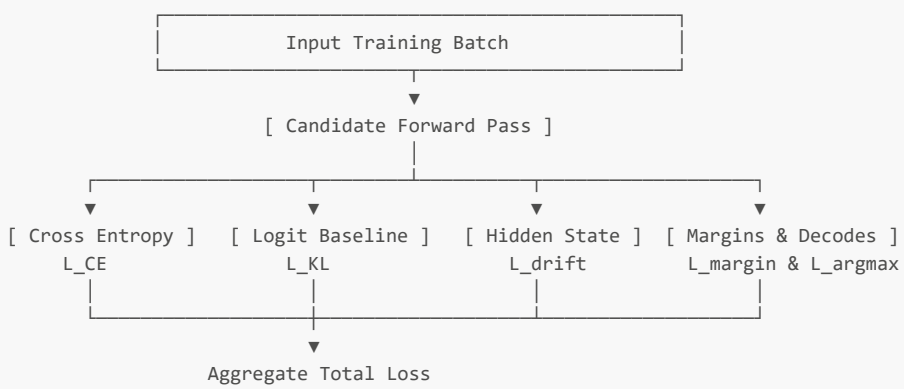
- **Layer 3 Bottleneck Importance:**

$$D_R(e_3) = -\frac{0.9397}{0.9627} \approx -0.9761$$

---

4. Feature Track (c): Advanced 6-term Regularization Loss Objective (CDL v2)

The CDL v2 objective balances output accuracy with representational stability. It combines cross-entropy with five auxiliary loss terms to optimize the network without over-steering.



We compute each of the six loss components for a single parameter update step. We set the active regularization weights to:

$$\lambda_{\text{drift}} = 0.05,\ \lambda_{\text{kl}} = 0.10,\ \lambda_{\text{margin}} = 0.05,\ \lambda_{\text{argmax}} = 0.10,\ \lambda_{\text{wrong}} = 0.10$$

Term-by-Term Calculations

Term 1: Task Cross-Entropy Loss ($\mathcal{L}_{\text{CE}}$)

This is the standard cross-entropy loss over the target token completion. Given our target probability $P_{\text{candidate}}(\text{"paris"} \mid x) = 0.7086$:

$$\mathcal{L}_{\text{CE}} = -\ln(0.7086) \approx 0.3444$$

Term 2: PCRF Structural Drift Loss ($\mathcal{L}_{\text{drift}}$)

This term penalizes representation drift between our candidate model and a frozen reference baseline model. It scales the cosine distance at each layer by its empirical derivative weight (Δ_l):

$$\mathcal{L}_{\text{drift}} = \sum_{l=0}^{3} \Delta_l \cdot (1.0 - \text{Cosine\_Similarity}(H_{l,\text{cand}},\ H_{l,\text{ref}}))$$

Suppose our hidden state cosine similarities are: Layer 0 = 0.999, Layer 1 = 0.991, Layer 2 = 0.985, Layer 3 = 0.970.

$$\mathcal{L}_{\text{drift}} = [0.2330 \cdot (1 - 0.999)] + [0.0510 \cdot (1 - 0.991)] + [0.0 \cdot (1 - 0.985)] + [0.1160 \cdot (1 - 0.970)]$$

$$\mathcal{L}_{\text{drift}} = 0.00023 + 0.00046 + 0.0 + 0.00348 \approx 0.00417$$

Term 3: KL Divergence from Baseline Logits ($\mathcal{L}_{\text{KL}}$)

This term keeps the candidate model's overall probability distribution close to the reference model to prevent catastrophic forgetting. Given our target logits z_{cand} and baseline logits z_{ref}:

$$z_{\text{cand}} = [6.8,\ 5.9,\ 1.0,\ 0.3,\ -1.4,\ -2.8,\ -4.0,\ -4.8]$$

$$z_{\text{ref}} = [8.2,\ 5.4,\ 1.1,\ 0.2,\ -1.5,\ -3.0,\ -4.2,\ -5.0]$$

We convert these to probability distributions Q and P using Softmax:

$$Q_{\text{cand}} = [0.7086,\ 0.2882,\ 0.0021,\ 0.0010,\ 0.0001,\ 0.0000,\ 0.0000,\ 0.0000]$$

$$P_{\text{ref}} = [0.9416,\ 0.0573,\ 0.0008,\ 0.0003,\ 0.0000,\ 0.0000,\ 0.0000,\ 0.0000]$$

$$\mathcal{L}_{\text{KL}} = \sum_{i=1}^{|\mathcal{V}|} P_{\text{ref},i} \ln\left(\frac{P_{\text{ref},i}}{Q_{\text{cand},i}}\right)$$

$$\mathcal{L}_{\text{KL}} = \left[0.9416 \ln\left(\frac{0.9416}{0.7086}\right)\right] + \left[0.0573 \ln\left(\frac{0.0573}{0.2882}\right)\right] + \dots$$

$$\mathcal{L}_{\text{KL}} = [0.9416 \times 0.2843] + [0.0573 \times -1.6154] \approx 0.2677 - 0.0926 \approx 0.1751$$

Term 4: Target Token Margin Loss ($\mathcal{L}_{\text{margin}}$)

This hinge loss forces the target token's logit past the second-place competing token's logit by a configured safety margin ($\gamma = 0.10$):

$$\mathcal{L}_{\text{margin}} = \max\left(0.0,\ \gamma - (P_{\text{cand}}(\text{target}) - P_{\text{cand}}(\text{second}))\right)$$

- Our target token `"paris"` has probability 0.7086.
- Our second-place competitor `"london"` has probability 0.2882.

$$\mathcal{L}_{\text{margin}} = \max\left(0.0,\ 0.10 - (0.7086 - 0.2882)\right) = \max(0.0,\ -0.3204) = 0.0000$$

Since the candidate model's prediction margin (0.4204) already exceeds the safety threshold, this penalty evaluates to zero.

Term 5: Argmax Stability Loss ($\mathcal{L}_{\text{argmax}}$)

This term applies a penalty if the candidate model's argmax prediction deviates from a baseline prediction that was originally correct.

- Since our baseline prediction was correct (`"paris"`), and our candidate prediction is also correct (highest logit is 6.8, which maps to `"paris"`):

$$\mathcal{L}_{\text{argmax}} = 0.0000$$

Term 6: High-Confidence Wrong Penalty ($\mathcal{L}_{\text{wrong}}$)

This term penalizes the candidate model if it generates incorrect completions with high confidence ($P_{\text{cand}} > 0.50$):

$$\mathcal{L}_{\text{wrong}} = \max(0.0,\ P_{\text{cand}}(\text{incorrect}) - 0.50)$$

Since the candidate model's top prediction is correct, this term is inactive:

$$\mathcal{L}_{\text{wrong}} = 0.0000$$

Aggregate Total Loss Summation

We sum all six components using our regularization weights to find the final training loss for this step:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + (\lambda_{\text{drift}} \cdot \mathcal{L}_{\text{drift}}) + (\lambda_{\text{kl}} \cdot \mathcal{L}_{\text{kl}}) + (\lambda_{\text{margin}} \cdot \mathcal{L}_{\text{margin}}) + (\lambda_{\text{argmax}} \cdot \mathcal{L}_{\text{argmax}}) + (\lambda_{\text{wrong}} \cdot \mathcal{L}_{\text{wrong}})$$

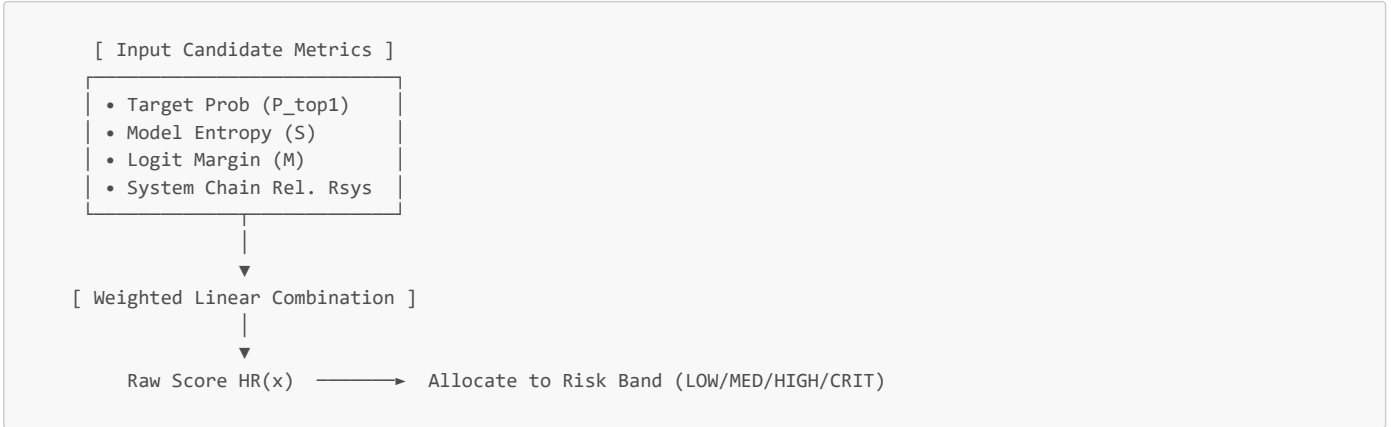
$$\mathcal{L}_{\text{total}} = 0.3444 + (0.05 \cdot 0.00417) + (0.10 \cdot 0.1751) + 0.0 + 0.0 + 0.0$$

$$\mathcal{L}_{\text{total}} \approx 0.3444 + 0.00021 + 0.01751 \approx 0.3621$$

---

5. Hallucination Risk Scoring $HR(x)$

The Hallucination Risk Score (HR) is a continuous, normalized metric (0.0 to 1.0) that measures the likelihood that a model's completion is a hallucination.



We evaluate the hallucination risk of a wrong candidate prediction using six weighted components:

$$\text{Weights: } w_1 = 0.15\ (\text{entropy}),\ w_2 = 0.15\ (\text{margin}),\ w_3 = 0.10\ (\text{KL}),\ w_4 = 0.20\ (\text{structural}),\ w_5 = 0.15\ (\text{instability}),\ w_6 = 0.25\ (\text{conf})$$

Step-by-Step Trace

Step 1: Input Metrics Configuration

Consider a scenario where the candidate model generates an incorrect completion with high confidence:

- **Prediction Status:** Incorrect.
- **Top Token Probability (P_{top1}):** 0.7200 (High confidence, wrong answer).
- **Second Token Probability (P_{top2}):** 0.1800.
- **Logits:** $z_{\text{incorrect}} = 6.4$, $z_{\text{correct}} = 5.2$.
- **Model Entropy (S):** 0.8500.
- **KL Divergence (D_{KL}):** 0.4500.
- **System Reliability (R_{sys}):** 0.3511 (35.11% - severe decay).

Step 2: Component Normalization

Component 1: Entropy Risk

We normalize the model's entropy against the maximum possible entropy for our vocabulary ($\ln|\mathcal{V}| = \ln(8) \approx 2.0794$):

$$\text{Risk}_{\text{entropy}} = \frac{S}{\ln|\mathcal{V}|} = \frac{0.8500}{2.0794} \approx 0.4088$$

Component 2: Margin Risk

We compute the logit margin risk using a standard sigmoid function:

$$\text{Margin } M = z_{\text{incorrect}} - z_{\text{correct}} = 6.4 - 5.2 = 1.2$$

$$\text{Risk}_{\text{margin}} = \frac{1}{1 + e^{-M}} = \frac{1}{1 + e^{-1.2}} \approx 0.7685$$

Component 3: KL Risk

We normalize the logit divergence penalty:

$$\text{Risk}_{\text{KL}} = 1.0 - e^{-D_{\text{KL}}} = 1.0 - e^{-0.4500} \approx 0.3624$$

Component 4: Structural Risk

This represents representational decay across the sequential layers:

$$\text{Risk}_{\text{structural}} = 1.0 - R_{\text{sys}} = 1.0 - 0.3511 = 0.6489$$

Component 5: Layer Instability Risk

We measure this as the average structural entropy across the network:

$$\text{Risk}_{\text{instability}} = 1.0 - e^{-S_{\text{total}}} = 1.0 - e^{-0.0621} \approx 0.0602$$

Component 6: High-Confidence Wrong Risk

Since the model generated an incorrect completion with high confidence ($P > 0.50$):

$$\text{Risk}_{\text{HCW}} = 1.0000$$

Step 3: Weighted Summation

We compute the raw Hallucination Risk Score (HR):

$$HR(x) = (w_1 \cdot \text{Risk}_{\text{entropy}}) + (w_2 \cdot \text{Risk}_{\text{margin}}) + (w_3 \cdot \text{Risk}_{\text{KL}}) + (w_4 \cdot \text{Risk}_{\text{structural}}) + (w_5 \cdot \text{Risk}_{\text{instability}}) + (w_6 \cdot \text{Risk}_{\text{HCW}})$$

$$HR(x) = (0.15 \cdot 0.4088) + (0.15 \cdot 0.7685) + (0.10 \cdot 0.3624) + (0.20 \cdot 0.6489) + (0.15 \cdot 0.0602) + (0.25 \cdot 1.0000)$$

$$HR(x) = 0.0613 + 0.1153 + 0.0362 + 0.1298 + 0.0090 + 0.2500 = 0.6016$$

Step 4: Risk Band Classification

We map the raw score (0.6016) to our configured safety bands:

- `LOW` Risk: $HR < 0.30$
- `MEDIUM` Risk: $0.30 \leq HR < 0.55$
- `HIGH` **Risk:** $0.55 \leq HR < 0.75$

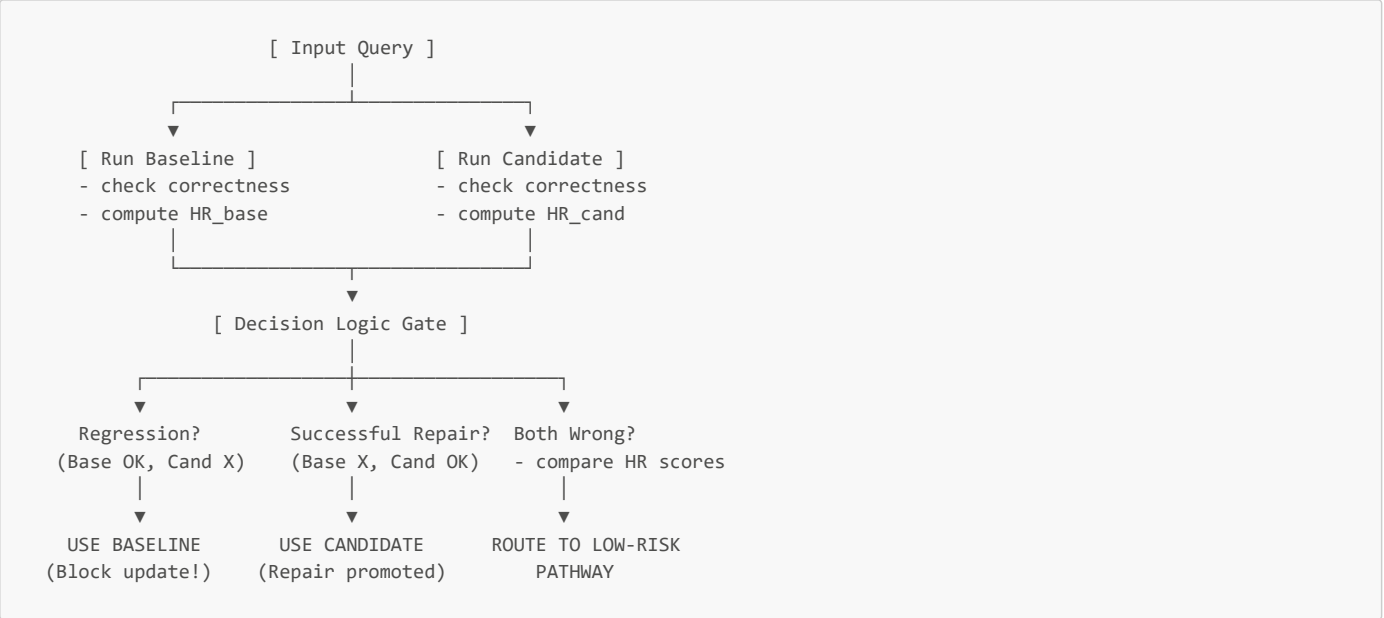
- CRITICAL Risk: $HR \geq 0.75$

Our score fall within the HIGH risk band. The system will flag this completion and route the query to a fallback RAG pipeline to prevent a hallucination from reaching the user.

---

6. Zero-Regression Repair Router

The Zero-Regression Repair Router acts as a gateway in production, choosing whether to route a user's query to the baseline model or the candidate model.



The router evaluates predictions on a case-by-case basis using a strict truth table to prevent regressions:

Decision Matrix Truth Table

Case	Baseline Correct?	Candidate Correct?	Decision Path	Action Taken
1	Yes	Yes	**Use Candidate**	The candidate's output is accepted. The router can also default to whichever model has the lower HR score or entropy.
2	Yes	No	**Use Baseline**	**Regression Blocked.** The candidate is rejected because it broke a previously working concept. The router defaults back to the baseline.
3	No	Yes	**Use Candidate**	**Repair Accepted.** The candidate successfully corrected a baseline error. The candidate's output is promoted.
4	No	No	**Lowest Risk Path**	Both models are incorrect. The router compares their Hallucination Risk (HR) scores and defaults to whichever model has the lower score.

---

7. Gating & Decision Directives

The final step of the pipeline aggregates all scorecard metrics into a single weighted index to determine the deployment directive.

Adoption Index Formula

$$\text{Index} = (0.20 \cdot \text{Score}_{\text{deriv}}) + (0.20 \cdot \text{Score}_{\text{curr}}) + (0.30 \cdot \text{Score}_{\text{struct}}) + (0.30 \cdot \text{Score}_{\text{reg}})$$

Using the scorecard values from our Qwen-0.5B evaluation run:

$$\text{Score}_{\text{deriv}} = 3.7,\ \text{Score}_{\text{curr}} = 75.6,\ \text{Score}_{\text{struct}} = 35.1,\ \text{Score}_{\text{reg}} = 60.0$$

$$\text{Index} = (0.20 \cdot 3.7) + (0.20 \cdot 75.6) + (0.30 \cdot 35.1) + (0.30 \cdot 60.0)$$

$$\text{Index} = 0.74 + 15.12 + 10.53 + 18.00 = 44.39$$

Deployment Directive Classification

- **`Index` $\geq$ 80.0:** `SAFE_TO_APPLY_GLOBAL` (🟢 Green Tier - Promote candidate everywhere).
- **50.0 $\leq$ `Index` < 80.0:** `SAFE_TO_APPLY_ROUTER_ONLY` (🟡 Yellow Tier - Deploy candidate behind the Zero-Regression Router).
- **`Index` < 50.0:** `REJECT_ADOPTION` (🔴 Red Tier - Candidate is rejected due to high structural drift and over-steering risk).

With an index score of 44.39, this candidate model is **rejected**. The framework protects the production system by blocking the update, exporting the failure traces to the debugger, and reverting the server to the baseline parameters.